

Introduction to Docker and deal.II Setup

Goal of the Lecture

In this lecture, we will:

- Understand what Docker is
- Create and manage a container
- Compile and run a simple C++ program using deal.II

Key idea: We use Docker to ensure everyone has the same environment.

What is Docker?

Docker allows us to run applications inside **containers**.

- A container is like a small computer inside your computer
- It contains all required tools and libraries

Analogy: A container is like a lunchbox with everything packed inside.

1.1 Install Docker

Install Docker Desktop from the official website.

1.2 Pull the Docker Image

```
docker pull pcafrica/adv_sci_comp
```

We are downloading a ready-made scientific environment.

- docker = the program
- pull = download
- pcafrica/adv_sci_comp = the image name

```
docker image ls
```

This lists all downloaded images.

1.3 Create a Container

```
docker run --name adv_sci_comp -v /path/to/host/folder:/home/dealii/  
shared-folder -it -d pcafrica/adv_sci_comp
```

For example, after creating **mkdir -p /shared-folder**

```
docker run --name adv_sci_comp \  
-v ~/shared-folder:/home/dealii/shared-folder \  
-it -d pcafrica/adv_sci_comp
```

- **docker run**: create and start a container
- **--name adv_sci_comp**: name of the container
- **-v**: connects folders

Important:

- Left path = your computer
- Right path = container

This allows file sharing between host (your real computer/laptop) and container (the isolated Docker environment). Normally, the container cannot see your files and your computer cannot see container files.

Optional Flags:

- **-it**: interactive terminal
- **-d**: run in background

Start container

```
docker start adv_sci_comp
```

This turns on the container.

1.4 Use the Container

```
docker exec -it adv_sci_comp /bin/bash
```

- **exec** = run command inside container
- **/bin/bash** = open terminal

Now we are inside the container.

Exit container

```
exit
```

Return to your computer.

Check containers

```
docker ps -a
```

Shows all containers and their status.

Stop container

```
docker stop adv_sci_comp
```

Important Warning:

Do NOT use docker run again, otherwise you create a new container every time.

Remove container

```
docker rm <container-name>
```

1.5 Editing Files

- Use an editor like VS Code
- Edit files on your computer
- Compile inside Docker

Key idea: Edit outside, run inside.

2. Test the Installation

Now, we write a very small program whose only job is to say: ‘Everything is working correctly.’

2.1 Create test.cpp

```
#include <deal.II/base/config.h>
#include <iostream>

int main()
{
    std::cout << "Successfully loaded deal.II." << std::endl;
    return 0;
}
```

- **#include:** include libraries
- deal.II = scientific library (We are telling the program: ‘We want to use deal.II’)
- iostream = printing (This allows the program to print something on the screen)
- main() = starting point (Think of this as the ‘start button’ of the program)
- std::cout = print to screen
- std::endl = new line (This moves to the next line after printing)
- return 0 = success

2.2 Create CMakeLists.txt

This file tells the computer how to build and compile our program. If the C++ file is the recipe, then this file is the cooking instructions for the machine.

```
cmake_minimum_required(VERSION 3.13.4)

project(test_project)

find_package(deal.II 9.6.0 REQUIRED
  HINTS ${DEAL_II_DIR} ../ ../.. $ENV{DEAL_II_DIR}
)

deal_ii_initialize_cached_variables()

add_executable(test test.cpp)
deal_ii_setup_target(test)
```

- `cmake_minimum_required`: required version
- `project`: project name (This gives a name to our project)
- `find_package`: locate deal.II (This tells the system to find the deal.II library)
- `REQUIRED`: this is mandatory; without it, just stop
- `deal_ii_initialize_cached_variables()`: This prepares internal settings for deal.II
- `add_executable(test test.cpp)`: create program called ‘test’ using the file test.cpp
- `setup_target`: link libraries (It links everything together so the program can run)

2.3 Build and Run

```
mkdir build
cd build
cmake ..
make -jN
./test
```

- `mkdir build`: create folder (we keep all compiled files separate from our source code)
- `cmake ..`: configure project (this reads the CMakeLists.txt file in parent folder and prepares everything for compilation)
- `make -jN`: compile, $N = \text{CPU cores}$ (builds the program)
- `./test`: run program

For normal code changes, `make -jN` is enough. You only rerun `cmake` when the build configuration changes.

Expected Output

```
Successfully loaded deal.II.
```